KevRouter: Decision Model Cost-Aware Router

Blake de Haas
blakedehaas@github.com

Abhinav Dugar
dugarabhinav2001@gmail.com

November 10, 2026

Abstract

[TODO: Abstract to be completed upon final benchmark convergence.]

1 Introduction

[TODO: Introduction to be completed at project conclusion.]

2 Related Work

[TODO: Related Work reviewing LLM cascade routing, mixture-of-agents, and calibration.]

3 Dataset and Benchmark Suite

To train and rigorously evaluate autonomous routing policies across heterogeneous workloads, our corpus is curated from LLMRouterBench, comprising 12,444 multi-model evaluated instances spanning 10 established benchmarks. The dataset encompasses broad factual recall, multi-step competition mathematics, professional college-level reasoning, complex code generation, and interactive agentic environments.

3.1 Benchmark Composition and Evaluation Regimes

The benchmark suite is partitioned into two distinct evaluation regimes:

1. **In-Distribution ($\mathcal{D}_{\text{ID}}$)**: 9 diverse benchmarks comprising 12,166 unique queries used for model fine-tuning, temperature calibration, hyperparameter optimization, and held-out in-distribution test evaluation.

2. **Out-of-Distribution ($\mathcal{D}_{\text{OOD}}$)**: τ^2-Bench (278 queries), held out entirely from training and validation to evaluate zero-shot transfer to multi-turn agentic tool-use environments.

As summarized in Table 1, the benchmark suite exhibits high difficulty variance. Highly solvable benchmarks like ArenaHard (97.7%) and AIME (96.7%) provide large cost-arbitrage opportunities for lightweight models. Conversely, HLE (51.0%) and SWE-Bench (64.4%) represent frontier capability bottlenecks where cost-effective models fail and routing must escalate to high-capacity reasoning flagships.

Table 1: Benchmark composition, domain coverage, solvability rate, and stratified sample counts.

Benchmark	Target Domain / Capability	Solvability	Train	Val	Test ID	Total
SimpleQA	Open-Domain Factuality & Short QA	84.3%	3,461	432	433	4,326
MMLU-Pro	College-Level Multi-Disciplinary Reasoning	95.0%	2,400	300	299	2,999
HLE	Humanity's Last Exam (Frontier Multimodal)	51.0%	1,728	218	212	2,158
LiveCodeBench	Algorithmic Problem Solving & Coding	92.0%	845	105	105	1,055
ArenaHard	Multi-Turn Instruction Following	97.7%	600	76	73	749
SWE-Bench	End-to-End Repository Bug Resolution	64.4%	398	50	52	500
GPQA	Graduate-Level Biology, Physics, Chem	97.0%	159	18	21	198
LiveMathBench	Fresh Competition Mathematics	84.3%	97	11	13	121
AIME	Elite Olympiad Mathematics	96.7%	50	5	5	60
In-Distribution	**Combined Primary Benchmarks**	**82.0%**	**9,738**	**1,215**	**1,213**	**12,166**
τ^2-Bench (*OOD*)	Multi-Turn Airline/Retail Agentic Tool Use	93.5%	—	—	278	278

Table 2: Candidate model pool, execution pricing (c_m), in-distribution standalone accuracy, and cost-optimal market share across the ground-truth dataset.

Model (m)	Provider / Family	Price / Query (c_m)	ID Accuracy	Optimal Share
`deepseek-v3-0324`	DeepSeek	\$0.002	41.28%	8.86%
`gpt-5-chat`	OpenAI	\$0.002	46.76%	3.88%
`deepseek-v3.1-terminus`	DeepSeek	\$0.004	43.49%	10.16%
`qwen3-235b-a22b-2507`	Alibaba Cloud	\$0.007	53.30%	**47.96%**
`intern-s1`	OpenGVLab	\$0.010	36.91%	2.00%
`kimi-k2-0905`	Moonshot AI	\$0.012	44.02%	4.59%
`claude-sonnet-4`	Anthropic	\$0.016	37.74%	0.48%
`gemini-2.5-flash`	Google DeepMind	\$0.020	42.68%	6.17%
`qwen3-235b-a22b-thinking-2507`	Alibaba Cloud	\$0.021	52.12%	4.58%
`deepseek-r1-0528`	DeepSeek	\$0.050	47.33%	1.47%
`glm-4.6`	Zhipu AI	\$0.055	43.75%	1.59%
`gpt-5`	OpenAI	\$0.150	57.64%	5.33%
`gemini-2.5-pro`	Google DeepMind	\$0.250	**58.80%**	2.92%

3.2 Candidate Model Pool (13-Class Action Space)

The router's destination action space $\mathcal{M}$ contains $|\mathcal{M}| = 13$ contemporary foundation models. The candidate endpoints span an execution cost disparity exceeding 125×, ranging from \$0.002 to \$0.250 per query.

As reported in Table 2:

- **Single Best Model**: `gemini-2.5-pro` achieves the highest individual accuracy across $\mathcal{D}_{\text{ID}}$ at 58.80%, but incurs the highest unit cost (\$0.250/query).
- **Cheapest Baseline**: `deepseek-v3-0324` achieves 41.28% accuracy at \$0.002/query (a 17.52% accuracy gap).
- **Workhorse Foundation**: `qwen3-235b-a22b-2507` captures 47.96% of all optimal ground-truth instances, serving as the dominant balance point between cost (\$0.007) and accuracy (53.30%).
- **Theoretical Oracle (Cheapest-Correct)**: Directing each solvable query to its lowest-cost successful model achieves an aggregate accuracy of **81.97%** at an average cost of only **\$0.0122 / query**—reducing expenditure by 20.5× compared to the flagship baseline while simultaneously improving task accuracy by +23.17%.

3.3 Ground-Truth Label Formulation

For each query instance x_i, upstream execution logs record the binary task correctness $y_{i,m} \in \{0, 1\}$ and execution cost $c_{i,m}$ across all models $m \in \mathcal{M}$.

Deterministic Optimal Model (m_i^*) The primary hard label represents the cheapest model that correctly solves x_i:

$$m_i^* = \arg \min_{m \in \mathcal{M}_{\text{correct}}(x_i)} c_{i,m}, \quad \text{where} \quad \mathcal{M}_{\text{correct}}(x_i) = \{m \in \mathcal{M} \mid y_{i,m} = 1\}. \tag{1}$$

If all 13 models fail on x_i ($\mathcal{M}_{\text{correct}} = \emptyset$, representing 18.03% of in-distribution instances), the label defaults to the lowest-cost model (`deepseek-v3-0324`), teaching the policy to avoid squandering expensive compute on unsolvable inputs.

Cost-Sensitive Soft Target Distribution (q_i) Because multiple models frequently solve a query with negligible price differences, training strictly against a one-hot label introduces artificial gradient penalties. We construct an exponential cost-discounted soft probability target distribution:

$$q(m \mid x_i) = \begin{cases} \frac{\exp\left(-\beta \cdot (c_{i,m} - c_{i,m^*})\right)}{\sum_{m' \in \mathcal{M}_{\text{correct}}(x_i)} \exp\left(-\beta \cdot (c_{i,m'} - c_{i,m^*})\right)}, & \text{if } m \in \mathcal{M}_{\text{correct}}(x_i), \\ 0, & \text{if } m \notin \mathcal{M}_{\text{correct}}(x_i), \end{cases} \tag{2}$$

where $\beta = 20.0$ modulates the cost penalty steepness, concentrating mass on the cheapest correct candidates while softly crediting near-optimal alternatives.

3.4 Hierarchical Doubly Stratified Partitioning

To prevent covariate shift and preserve identical domain and difficulty distributions across partitions, we implement a **Hierarchical Doubly Stratified Sampling Strategy** (seed = 42).

Rather than stratifying over a single attribute, the corpus is partitioned across the joint distribution of **Benchmark Domain** $\times$ **Target Model Class** ($D \times M$):

1. **Primary Stratum**: The dataset is segregated by benchmark dataset D (AIME, GPQA, SWE-Bench, etc.).
2. **Secondary Stratum**: Within each benchmark D, queries are partitioned into 13 bins by their optimal routing target m^*.
3. **Deterministic Shuffling**: To ensure cross-platform reproducibility against non-deterministic filesystem orderings, records within each (D, m^*) cell are sorted by the SHA-256 hash of their whitespace-normalized prompt text:

   $$h_i = \text{SHA-256}(\text{normalize}(x_i)), \tag{3}$$

   and then pseudorandomly shuffled with seed 42.
4. **Split Allocation**: Each cell is partitioned 80.0% to Train (9,738 queries), 10.0% to Validation (1,215 queries), and 10.0% to In-Distribution Test (1,213 queries).

This ensures that every individual benchmark maintains an exact 80/10/10 distribution, and every candidate model's target frequency is preserved without class starvation in small benchmarks.

3.5 Validation Sub-Partitioning and Leakage Mitigation

The 10% validation split ($\mathcal{D}_{\text{val}}$, 1,215 queries) is subdivided into two statistically disjoint sets:

- **Calibration Set ($N = 500$)**: Dedicated exclusively to post-hoc temperature scaling via Negative Log-Likelihood (NLL) minimization. Because temperature scaling optimizes a single 1D scalar parameter ($T \in \mathbb{R}^+$), 500 instances provide sufficient statistical power for convergence.

- **Development Set ($N = 715$)**: Dedicated to evaluating calibrated probabilities, conducting hyperparameter sweeps over loss weighting (μ), and tuning confidence cascading thresholds (τ).

Mitigation of Circular Validation Leakage Fitting temperature T on the same validation samples used to evaluate calibration metrics (ECE, Brier score) or tune decision thresholds introduces circular selection bias: the temperature memorizes the sample residuals, yielding artificially optimistic calibration estimates. Maintaining strictly disjoint calibration and development sets guarantees that decision thresholds are tuned on genuinely unobserved probabilistic predictions.

3.6 Prompt Length Distribution and 95th-Percentile Truncation

Prompt lengths across the raw corpus exhibit extreme variance, spanning from 5 tokens up to 261,898 tokens (measured via Byte-Pair Encoding with `cl100k_base`).

Table 3: Empirical prompt length distribution across benchmarks (BPE tokens).

Benchmark	**p50 (Median)**	**p75**	**p90**	**p95**	**p99**	**Max**
SimpleQA	21	26	34	**40**	54	105
LiveMathBench	119	150	192	**221**	263	366
AIME	150	184	258	**280**	340	398
τ^2-Bench (*OOD*)	158	257	318	**319**	320	367
MMLU-Pro	197	273	382	**462**	643	1,265
GPQA	218	282	366	**462**	789	2,809
HLE	183	316	555	**847**	1,740	13,560
LiveCodeBench	526	667	814	**942**	1,209	1,639
ArenaHard	90	267	733	**1,391**	3,223	7,406
SWE-Bench	11,405	21,869	30,715	**46,665**	104,232	261,898
In-Distribution ($\mathcal{D}_{\text{ID}}$)	**136**	**287**	**604**	**1,219**	**22,117**	**261,898**

Informed Truncation Policy (1,024 Tokens) As detailed in Table 3, the empirical 95th percentile across all in-distribution queries is 1,219 tokens. Default short-text heuristics (e.g., 1,400 characters / $\approx$ 320 tokens) severed 18.96% of all instances, truncating essential function signatures in LiveCodeBench and problem descriptions in SWE-Bench.

We institute an empirical truncation threshold of **1,024 tokens** ($\approx 4,000$ characters):

- Exactly **94.58% of all in-distribution queries** (and 95.2% of the training split) are retained completely losslessly with zero truncation.
- 100% of SimpleQA, LiveMathBench, AIME, GPQA, τ^2-Bench, and MMLU-Pro queries remain intact.
- For long-tail outliers (SWE-Bench), the initial 1,024 tokens capture the full issue statement, expected behavior, and reproduction stack trace, while pruning multi-megabyte git patch diffs that would otherwise induce quadratic attention memory blowup during routing.

3.7 Dynamic Candidate Logit Masking

In the upstream release of τ^2-Bench, `gpt-5-chat` was omitted from evaluation. Rather than discarding the benchmark or altering the 13-class routing space, we implement **Dynamic Can-**

didate Logit Masking at inference time:

$$z'_m(x) = \begin{cases} z_m(x), & \text{if } m \in \mathcal{M}_{\text{evaluated}}(x), \\ -\infty, & \text{if } m \notin \mathcal{M}_{\text{evaluated}}(x). \end{cases} \tag{4}$$

Applying this mask prior to softmax normalization guarantees that unevaluated models receive zero probability mass, eliminating false-failure evaluation penalties while preserving architectural consistency.

4 Methods: Fine-Tuning Methodology

KevRouter frames cost-aware routing as a learned single-pass decision model. Rather than generating free-form routing text or running multi-agent debate loops, the router predicts a calibrated probability distribution over destination endpoints in a single forward pass.

4.1 Model Architecture: Backbone and PointerHead

The routing architecture comprises an open foundation transformer backbone (Qwen 2.5 / 3.5 architecture) augmented with a lightweight, multi-option PointerHead decision mechanism.

Input Formulation Given a query x, the input sequence is formatted into a state context followed by structured option tokens describing the 13 candidate models and their routing criteria:

$$S = [\text{State: } x \ ; \ \text{Options: } \langle \text{opt}_1 \rangle, \ldots, \langle \text{opt}_K \rangle \ ; \ \langle \text{decide} \rangle]. \tag{5}$$

PointerHead Mechanism Let $h_{\text{decide}} \in \mathbb{R}^d$ denote the transformer hidden state corresponding to the final $\langle \text{decide} \rangle$ token, and let $H_{\text{opts}} \in \mathbb{R}^{K \times d}$ denote the hidden states pooled at each candidate option boundary $\langle \text{opt}_k \rangle$.

The PointerHead applies linear projections to map query decisions and option keys into a shared projection subspace of dimension $d_p = 256$:

$$q = W_q h_{\text{decide}} \in \mathbb{R}^{d_p}, \tag{6}$$

$$k_m = W_k H_{\text{opts}}[m] \in \mathbb{R}^{d_p}, \quad \forall m \in \{1, \ldots, K\}, \tag{7}$$

where $W_q, W_k \in \mathbb{R}^{d_p \times d}$. The uncalibrated routing logit z_m for candidate model m is computed via scaled dot-product attention:

$$z_m = \frac{k_m^\top q}{\sqrt{d_p}}. \tag{8}$$

4.2 Parameter-Efficient Fine-Tuning (PEFT) via LoRA

To enable parameter efficiency and prevent catastrophic forgetting of general semantic representations, the transformer backbone is adapted using Low-Rank Adaptation (LoRA).

For each targeted weight matrix $W_0 \in \mathbb{R}^{d_1 \times d_2}$, the weight update is decomposed into low-rank matrices $B \in \mathbb{R}^{d_1 \times r}$ and $A \in \mathbb{R}^{r \times d_2}$:

$$W = W_0 + \frac{\alpha}{r} BA, \tag{9}$$

where $r = 16$ is the intrinsic adapter rank, $\alpha = 32$ is the scaling factor, and adapter dropout is set to $p_{\text{drop}} = 0.05$.

LoRA adapters are injected into:

- Attention projection layers: Query (W_q), Key (W_k), Value (W_v), and Output (W_o).

- Multi-Layer Perceptron (MLP) layers: Gate (W_{gate}), Up-projection (W_{up}), and Down-projection (W_{down}).

The original backbone weights W_0 remain frozen. The trainable parameters consist solely of the LoRA matrices $\{A, B\}$ and the PointerHead projection weights $\{W_q, W_k\}$, representing less than 1.5% of the total backbone parameter count.

4.3 Training Objective

The model is trained on $\mathcal{D}_{\text{train}}$ using a cost-sensitive soft cross-entropy objective against the target distribution $q(m \mid x_i)$ defined in Section 3.3:

$$\mathcal{L}_{\text{route}} = -\sum_{i=1}^{N} \sum_{m=1}^{|\mathcal{M}|} q(m \mid x_i) \log \left(\frac{\exp(z_{i,m})}{\sum_{j=1}^{|\mathcal{M}|} \exp(z_{i,j})} \right). \tag{10}$$

Minimizing $\mathcal{L}_{\text{route}}$ is equivalent to minimizing the Kullback-Leibler (KL) divergence $\mathbb{D}_{\text{KL}}(q_i \parallel p_i)$ between the empirical cost-sensitive target distribution and the model's predicted softmax distribution.

The 4B and 9B experiments use AdamW with weight decay 0.01 and a one-cycle learning-rate schedule. Two configurations were evaluated: one epoch at learning rate 2×10^{-5} and three epochs at 2×10^{-4}. Because both settings differ, their comparison does not isolate epoch count. The historical 0.8B results use one epoch; the exact training-record manifest is unavailable.

4.4 Post-Hoc Probability Calibration

Following training, the model's logits are calibrated using temperature scaling on the held-out calibration partition ($\mathcal{D}_{\text{cal}}$, $N = 500$):

$$\hat{p}_m(x) = \frac{\exp(z_m / T)}{\sum_{j=1}^{|\mathcal{M}|} \exp(z_j / T)}, \tag{11}$$

where the scalar temperature $T > 0$ is obtained by minimizing Negative Log-Likelihood via L-BFGS:

$$T^* = \arg\min_{T>0} \left[-\sum_{i \in \mathcal{D}_{\text{cal}}} \sum_{m=1}^{|\mathcal{M}|} q(m \mid x_i) \log \left(\frac{\exp(z_{i,m}/T)}{\sum_{j=1}^{|\mathcal{M}|} \exp(z_{i,j}/T)} \right) \right]. \tag{12}$$

The optimal temperature T^* is frozen into the deployed decision head, producing well-calibrated confidence scores.

4.5 Inference-Time Decision Rules

Given calibrated probabilities $\hat{p} \in \Delta^{|\mathcal{M}|-1}$ and known model pricing vectors $c \in \mathbb{R}^{|\mathcal{M}|}$, the router executes one of three decision policies:

1. Greedy Argmax Policy Directs the query to the endpoint with the highest estimated success probability:

$$m^*_{\text{argmax}} = \arg\max_{m \in \mathcal{M}} \hat{p}_m(x). \tag{13}$$

2. Gentle Utility Cost-Penalized Policy Penalizes expensive endpoints by trading marginal confidence against execution cost via regularizer $\mu \geq 0$:

$$m^*_{\text{utility}} = \arg\max_{m \in \mathcal{M}} \left[\hat{p}_m(x) - \mu \cdot c_m\right]. \tag{14}$$

Varying μ shifts routing mass according to the probability and fixed model cost proxies. For the 4B and 9B analysis, we sweep 503 values from zero through 100 on $\mathcal{D}_{\text{dev}}$ and select settings under prespecified average-cost budgets.

3. Confidence-Gated Cascading Policy Partitions $\mathcal{M}$ into a cost-effective tier $\mathcal{M}_{\text{cheap}}$ ($c_m \leq \$0.010$) and a frontier flagship tier $\mathcal{M}_{\text{flagship}}$. The router evaluates whether the top cost-effective candidate meets confidence threshold $\tau \in [0.60, 0.85]$:

$$m^*_{\text{cascade}} = \begin{cases} \arg\max_{m \in \mathcal{M}_{\text{cheap}}} \hat{p}_m(x), & \text{if } \max_{m \in \mathcal{M}_{\text{cheap}}} \hat{p}_m(x) \geq \tau, \\ \arg\max_{m \in \mathcal{M}_{\text{flagship}}} \hat{p}_m(x), & \text{otherwise.} \end{cases} \tag{15}$$

This rule does not guarantee accuracy preservation; its empirical accuracy and cost are evaluated below.

5 Experiments and Results

We evaluate KevRouter on the full LLMRouterBench suite across in-distribution and out-of-distribution benchmarks. In accordance with the evaluation hierarchy established in Section 4, our analysis prioritizes:

1. **Routing Accuracy (%):** Proportion of queries routed to an endpoint that produces a fully correct response (score ≥ 1.0).
2. **Optimal Match Rate (%):** Proportion of queries routed specifically to the Pareto-optimal (cheapest correct) model.
3. **Cost per Correct Answer ($/query):** Total expenditure incurred divided by the number of correct responses, measuring net economic efficiency.
4. **Routing Latency (ms):** Single-pass forward runtime on the local host.

5.1 Experimental Setup

The historical 0.8B model was trained and evaluated on an NVIDIA GeForce RTX 5080 (16GB VRAM, CUDA 13.5). The router utilizes the open 852M parameter `Qwen/Qwen3.5-0.8B-Base` architecture with low-rank adaptation ($r = 16, \alpha = 32$) and PointerHead projection ($d_p = 256$), comprising 11.3M trainable parameters. Training executes for one complete epoch over 9,193 admitted training records in `bfloat16` with gradient checkpointing, batch size 1, and 8 accumulation micro-batches (effective batch size 8). Total fine-tuning runtime was 104.97 minutes with peak VRAM utilization of 3.92 GB.

For the historical 0.8B run, the paper reports a fitted temperature $T^* = 0.6156$ (Brier score: 0.7149, ECE: 0.3066). The 4B and 9B experiments and their calibration procedures are described separately below. The original 0.8B record-level predictions and training manifest are unavailable for independent verification.

5.2 Comparative Benchmark Matrix

Table 4 reports the comprehensive performance comparison between the zero-shot base checkpoint (Kev-0.8B Base) and fine-tuned KevRouter under three inference decision rules: greedy calibrated ArgMax, Cost-Utility Bayes optimization ($\mu = 2.5$), and confidence-gated cascading ($\tau = 0.6$).

Table 4: Comprehensive performance matrix across validation, held-out in-distribution test, and out-of-distribution (τ^2-Bench) splits on LLMRouterBench. Best results per metric within each split are bolded.

Model	Decision Policy	Accuracy (%)	Optimal (%)	Avg Cost ($/query)	Cost / Correct ($/correct)	Latency (ms)
Validation Split ($\mathcal{D}_{\text{val}}$, $N = 1,215$)						
Kev-0.8B (Base)	ArgMax (Zero-Shot)	47.49%	3.46%	$0.00712	$0.01499	91.5
Kev-0.8B (Fine-Tuned)	Calibrated ArgMax	**59.42%**	**39.01%**	$0.01577	$0.02654	90.8
Kev-0.8B (Fine-Tuned)	Cost-Utility ($\mu = 2.5$)	54.07%	36.38%	**$0.00098**	**$0.00181**	90.8
Kev-0.8B (Fine-Tuned)	Cascading ($\tau = 0.6$)	**59.42%**	**39.01%**	$0.01577	$0.02654	90.8
Held-Out In-Distribution Test Split ($\mathcal{D}_{\text{test-ID}}$, $N = 1,213$)						
Kev-0.8B (Base)	ArgMax (Zero-Shot)	45.75%	3.63%	$0.00765	$0.01672	92.9
Kev-0.8B (Fine-Tuned)	Calibrated ArgMax	**56.72%**	**36.44%**	$0.01447	$0.02551	98.9
Kev-0.8B (Fine-Tuned)	Cost-Utility ($\mu = 2.5$)	53.09%	35.28%	**$0.00087**	**$0.00164**	98.9
Kev-0.8B (Fine-Tuned)	Cascading ($\tau = 0.6$)	**56.72%**	**36.44%**	$0.01447	$0.02551	98.9
Out-of-Distribution Transfer Split (τ^2-Bench, $N = 278$)						
Kev-0.8B (Base)	ArgMax (Zero-Shot)	15.47%	2.88%	$0.06859	$0.44341	**90.9**
Kev-0.8B (Fine-Tuned)	Calibrated ArgMax	**40.29%**	**25.18%**	**$0.04198**	**$0.10421**	95.7
Kev-0.8B (Fine-Tuned)	Cost-Utility ($\mu = 2.5$)	**40.29%**	**25.18%**	**$0.04198**	**$0.10421**	95.7
Kev-0.8B (Fine-Tuned)	Cascading ($\tau = 0.6$)	**40.29%**	**25.18%**	**$0.04198**	**$0.10421**	95.7

5.3 4B and 9B Experiments

We additionally evaluated pretrained Kev 4B and 9B checkpoints and fine-tuned each on LLMRouterBench using rank-16 LoRA and the trainable routing head on an NVIDIA A100 80GB GPU. The Qwen backbones remained frozen. One run per model used one epoch at 2×10^{-5}; a second used three epochs at 2×10^{-4}, with batch size one and gradient accumulation over eight records. Both configurations admitted 9,193 training records under the same Qwen-tokenizer context filter used by the current pipeline: 545 of 9,738 records were excluded, including all 398 SWE-Bench records. This matches the reported 0.8B count, but the historical record-level manifest is unavailable, so exact identity cannot be verified. The three-epoch jobs each completed in approximately 9 hours 40 minutes including calibration and evaluation.

The one-epoch runs fitted temperature against hard labels using the installed trainer's grid search; the three-epoch runs fitted temperature on 500 held-out calibration examples by minimizing soft-target negative log-likelihood with L-BFGS-B. A positive temperature does not alter the argmax ordering. For the three-epoch runs, calibrated probabilities support the cost and tier policies below. Each development and test evaluation scores only candidates with a recorded outcome; on OOD, this excludes `gpt-5-chat`. Such offline masking is not a guarantee of deployment-time availability. Routing accuracy is the fraction of queries for which the selected candidate has recorded score at least one; cost is the recorded downstream cost and excludes router inference cost.

Table 5 shows a substantial ID improvement from fine-tuning each larger router, but no clear accuracy advantage from increasing router size. The three-epoch 4B and 9B models chose different destinations on 519 of 1,213 ID records; 4B alone was correct on 66 and 9B alone on 68. Their similar aggregate accuracy therefore does not imply identical routing behavior. On OOD, the one-epoch 4B and 9B models chose Qwen 235B on all 278 records, explaining their identical 40.29% accuracy.

5.4 Development-Selected Cost Policies

Using saved calibrated probabilities from the three-epoch runs, we swept 503 values of μ: zero, a logarithmic grid from 10^{-4} to 100, and 2.5. For each router and each prespecified development average-cost budget of $0.001, $0.002, $0.005, $0.010 and $0.020 per query, we selected the highest-accuracy eligible setting on the 715-record development set, breaking ties by lower cost

Table 5: Argmax results for historical 0.8B runs and the new 4B and 9B runs, compared with selecting Qwen 235B for every query. ID has 1,213 records and OOD has 278. Cost is the mean recorded ID downstream cost per query. The 0.8B rows are reported historical aggregates, without record-level predictions available for verification.

Router / policy	**ID accuracy**	**OOD accuracy**	**ID cost**
0.8B pretrained (historical)	45.75%	15.47%	$0.00765
0.8B fine-tuned, 1 epoch (historical)	56.72%	40.29%	$0.01447
4B pretrained	43.61%	41.01%	$0.00801
4B fine-tuned, 1 epoch	56.64%	40.29%	$0.01464
4B fine-tuned, 3 epochs	56.14%	37.77%	$0.01118
9B pretrained	45.01%	9.71%	$0.00959
9B fine-tuned, 1 epoch	56.80%	40.29%	$0.01421
9B fine-tuned, 3 epochs	56.31%	38.13%	$0.01115
Fixed Qwen 235B	53.42%	40.29%	$0.00083

Table 6: Development-selected cost-utility policies evaluated on the ID test set. Budgets refer to development mean cost, not test-time guarantees. All costs exclude router inference.

Router	**Dev. budget**	μ	**ID accuracy**	**ID cost**
4B	$0.001	73.7904	46.74%	$0.00089
4B	$0.002	15.2757	48.39%	$0.00152
4B	$0.005	0.7516	53.83%	$0.00475
4B	$0.010	0.0871	54.66%	$0.00901
4B	$0.020	0.0000	56.14%	$0.01118
9B	$0.001	73.7904	46.08%	$0.00092
9B	$0.002	1.5849	53.34%	$0.00184
9B	$0.005	1.2706	53.83%	$0.00239
9B	$0.010	0.1117	55.32%	$0.00940
9B	$0.020	0.0023	56.39%	$0.01114
Fixed Qwen 235B	—	—	53.42%	$0.00083
Fixed GPT-5	—	—	56.55%	$0.02933
Fixed Gemini 2.5 Pro	—	—	58.37%	$0.06039

and then lower μ. The fixed model cost proxies in the decision rule differ from the per-query recorded costs used to evaluate mean expenditure. An empirical budget on development is not a per-query guarantee and need not hold on other distributions. ID and OOD results had already been inspected before this analysis and are not fresh confirmation sets.

At the $0.005 development budget, the selected 9B policy reached 53.83% ID accuracy at $0.00239 per query, compared with 53.42% at $0.00083 for fixed Qwen. The 9B argmax policy reached 56.31% at $0.01115. Figure 1 places these operating points alongside all 13 fixed candidates; no single claim of cost efficiency applies independently of the desired accuracy and baseline.

5.5 Paired Uncertainty and Error Analysis

We computed paired 95% percentile bootstrap intervals from 5,000 resamples, clustering on query hash and using the same sampled clusters for both methods. Intervals describe sampling variation conditional on the saved trained models, not retraining variance or multiple-comparison uncertainty. For ID, three-epoch 9B argmax minus fixed Qwen was +2.89 percentage points (95% interval $[+0.66, +5.11]$): 9B alone succeeded on 112 records and Qwen alone on 77. For the 9B policy selected at the $0.005 development budget, the difference was +0.41 points ($[-1.57, +2.39]$), with 80 and 75 unique successes respectively. Three-epoch 4B

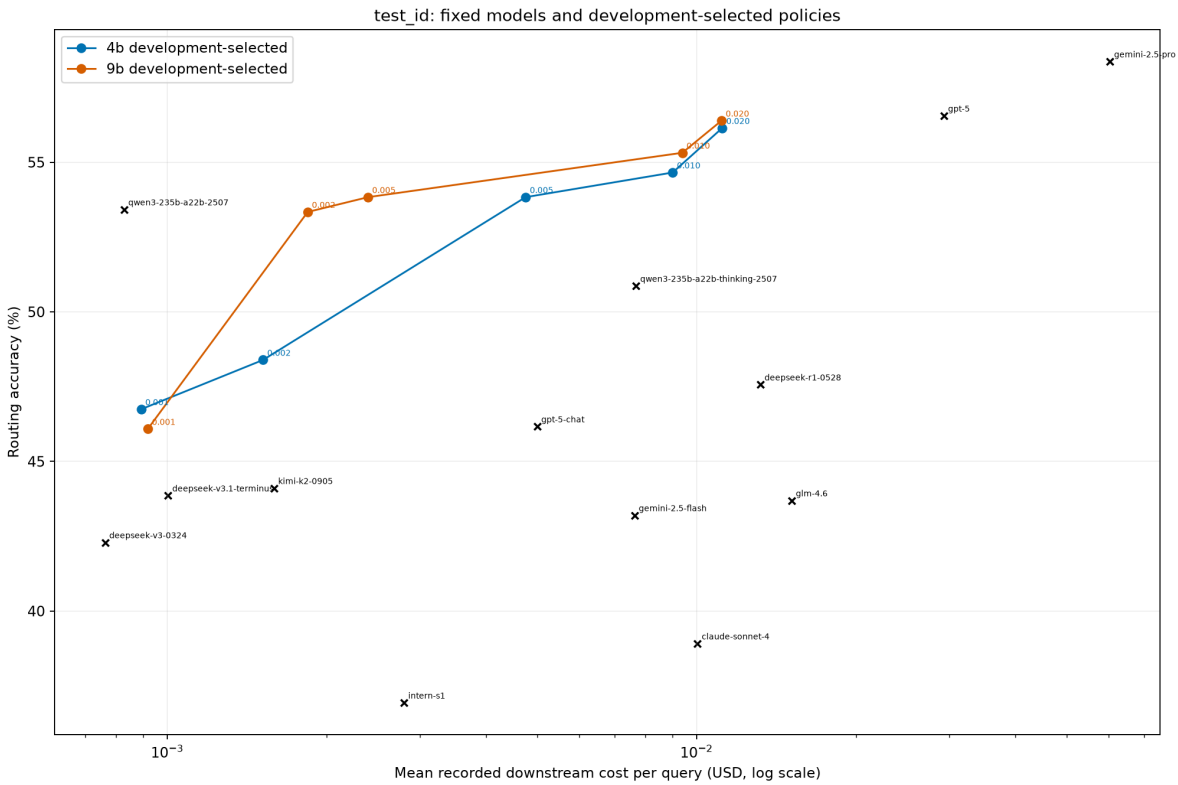


Figure 1: ID test accuracy and recorded downstream cost for every fixed-model baseline and the 4B/9B policies selected on development. Labels on router points denote development budgets in dollars per query. Lines connect evaluated settings and do not establish performance at interpolated points.

minus 9B argmax was -0.16 points ($[-2.06, +1.65]$). The one-epoch 4B minus 9B interval was $[-0.99, +0.66]$ points. The historical 0.8B aggregate lacks per-record predictions and cannot be included in paired comparisons.

Comparing three-epoch 9B argmax with fixed Qwen, ID correct counts were 32 versus 20 of 212 HLE records and 81 versus 72 of 105 LiveCodeBench records. They were 239 versus 240 of 299 MMLU-Pro records and 11 versus 11 of 52 SWE-Bench records. The three-epoch 9B router selected GPT-5 for 180 ID records, accounting for \$9.47 of its \$13.53 total recorded downstream cost; GPT-5 answered 69 of those selected records correctly. These queries are selected by the router and need not have the same difficulty as other queries. Breakdown by prompt length is similarly descriptive because length and benchmark type are correlated. Small per-benchmark groups and repeated inspection warrant caution.

5.6 Exploratory OOD Comparison

OOD contains 278 records with 167 distinct query hashes; bootstrap resampling clusters repeated query text, which may occur in distinct scenarios. The three-epoch 4B tier gate at $\tau = 0.6$ achieved 48.92% at \$0.10027 per query while selecting `kimi-k2-0905` on 273 records. Fixed Kimi achieved the same 48.92% at \$0.10036. Fixed `glm-4.6` achieved 53.96% at \$0.09931, exceeding the gate's observed accuracy at lower recorded cost. Thus, improvement over fixed Qwen alone is insufficient to establish an OOD routing benefit. Figure 2 compares all 12 available OOD fixed candidates with the unchanged development-selected 4B and 9B policies. These post hoc OOD comparisons are exploratory.

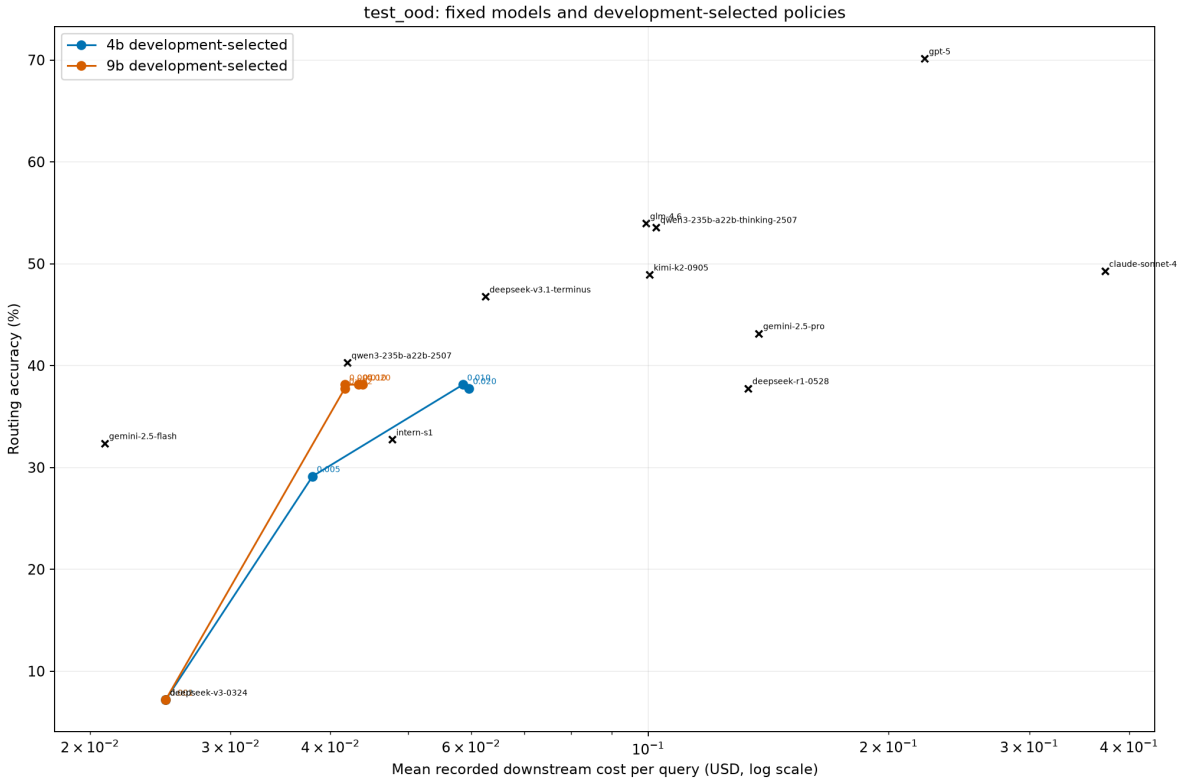


Figure 2: Exploratory OOD accuracy and recorded downstream cost for all available fixed models and the unchanged development-selected policies. Cost and accuracy reflect the benchmark's recorded outputs.

6 Discussion

Fine-tuning improves ID accuracy for both larger routers, but the observed 4B–9B difference is small and uncertain. The trade-off with fixed-model baselines is central: argmax routing improves ID accuracy over fixed Qwen at substantially greater recorded downstream cost, while the selected cheaper operating point does not establish an accuracy improvement. On OOD, the near-constant Kimi tier-gate choice illustrates why comparisons against several fixed candidates are necessary. The original 0.8B per-record outputs and exact training manifest are unavailable, the test sets have been examined repeatedly, and the two larger-model training configurations vary both epoch count and learning rate. These limitations prevent a controlled scaling or generalization claim.

7 Conclusion

On LLMRouterBench, fine-tuned 4B and 9B Kev routers achieve approximately 56–57% ID argmax accuracy, close to the previously reported 0.8B result. The 9B router improves ID accuracy over always choosing Qwen in a paired analysis, but at considerably higher recorded downstream cost. Development-selected cost policies and OOD fixed-model comparisons show that a routing benefit depends on the operating point and baseline. Independent held-out evaluation and controlled training ablations are needed to assess stability and generalization.